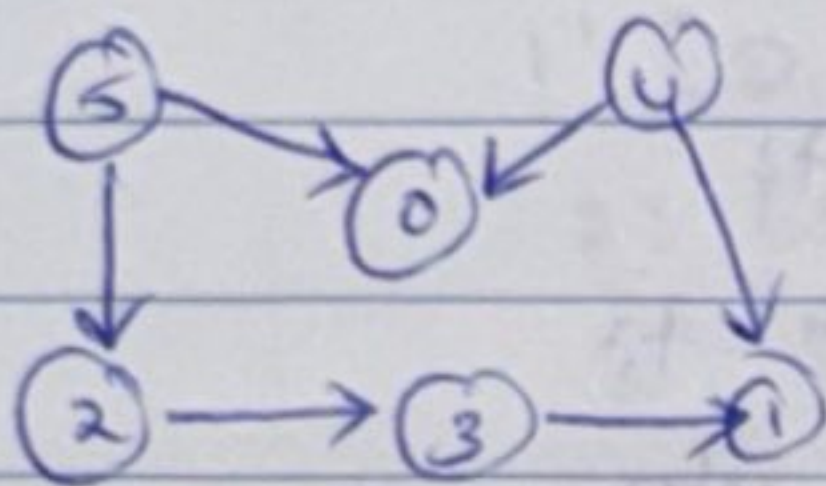


Topological sort

Linear ordering of vertices such that

if there is an edge between u & v

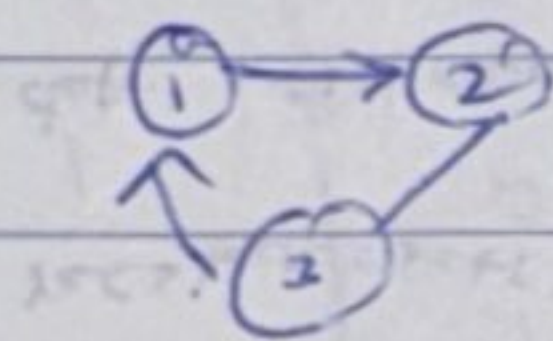
u appear before v in that ordering.



5 → 2 3 → 1
5 4 2 3 1 0 [5 before 2, 3 before 1]
4 5 2 3 1 0 [5 before 2, 4 before 1]

TS is applicable to Directed Acyclic Graph (DAG)

because



Cycle

3 → 1
1 2 3 4 [3 before 1 x]

use case Task scheduling, Course prerequisites.

with DFS:-

def top-sort-dfs(graph):

visited = set()

stack = []

def dfs(node):

visited.add(node)

for neighbour in graph[node]:

if neigh not in visited:

dfs(neigh)

stack.append(node)

// add node after visiting all neighbours

for node in graph:

if node not in visited

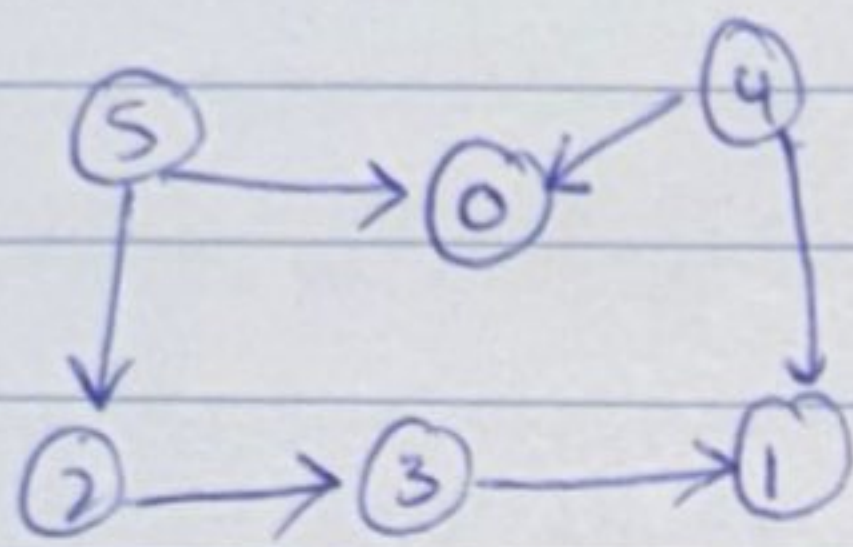
dfs(node)

return stack[::-1]

Kahn's Algorithm:

BFS	DFS
Queue	Stack
✓ cycle detect is easy	needs extra logic

1. Compute indegree (incoming edges)
2. Start with nodes having in-degree = 0
3. Remove them & reduce neighbors in-degree.



Indegree = { 5: 0, 4: 0, 2: 1, 3: 1, 1: 2, 0: 2 }

Step	Queue	Pop	Changes in in-degree	Queue After	To Do
1	[5, 4]	5	2: 1 → 0, 0: 2 → 1	[4, 2]	5
2	[4, 2]	4	0: 1 → 0, 1: 2 → 1	[2, 0]	5, 4
3	[2, 0]	2	3: 1 → 0	[0, 3]	5, 4, 2
4	[0, 3]	0	-	[3]	5, 4, 2, 0
5	[3]	3	1: 1 → 0	[1]	5, 4, 2, 0, 3
6	[1]	1	-	[]	5, 4, 2, 0, 3, 1

```
def top_sort_khan(graph):
```

```
    indegree = {node: 0 for node in graph}
```

```
    for node in graph:
```

```
        for neigh in graph[node]:
```

```
            indegree[neigh] += 1
```

```
    // Step 1: Add nodes with indegree 0 to queue
```

```
    queue = deque()
```

```
    for node in indegree:
```

```
        if indegree[node] == 0:
```

```
            queue.append(node)
```

```
    top_order = []
```

```
    while queue:
```

```
        node = queue.popleft()
```

```
        top_order.append(node)
```

```
        // reduce indegree neighbour
```

```
        for neigh in graph[node]:
```

```
            indegree[neigh] -= 1
```

```
            if indegree[neigh] == 0:
```

```
                queue.append(neigh)
```

```
    // Step 2: check for cycle
```

```
    if len(top_order) != len(graph):
```

```
        return "cycle detected"
```

```
    return top_order
```


3. Course schedule:-

→ take '0' list

numCourses = 2, prerequisites = [[1,0]] to take Course 1; you should
"detect cycle" using topological BFS (Kahn's).

1. Build graph

2. Add Course with no prerequisites

3. process Queue

4. check if all Course processed.

```
def canFinish(numCourses, prerequisites):
```

```
    graph = {i: [] for i in range(numCourses)}
```

```
    indegree = [0] * numCourses
```

```
    for course, prereq in prerequisites:
```

```
        graph[prereq].append(course)
```

```
        indegree[course] += 1
```

```
    queue = deque()
```

```
    for i in range(numCourses):
```

```
        if indegree[i] == 0:
```

```
            queue.append(i)
```

```
    completed = 0
```

```
    while queue:
```

```
        node = queue.popleft()
```

```
        completed += 1
```

```
        for neighbour in graph[node]:
```

```
            indegree[neighbour] -= 1
```

```
            if indegree[neighbour] == 0:
```

```
                queue.append(neighbour)
```

```
    return completed == numCourses
```


Course Schedule II:-

Instead of just checking True/False, return one valid order.

instead of completed = 0, use order = []

while queue:

node = queue.popleft()

order.append(node)

for neigh in graph[node]:

indegree[neigh] -= 1

if indegree[neigh] == 0:

queue.append(neigh)

if len(order) == numCourses:

return order

else

return [] // Cycle exists.

if there is cycle 0 → 1 → 2 → 0

start	Queue	order	Action
start	[0]	[]	
1	[]	[0]	process 0
2	empty	[0]	stop & len != count

Node	indegree
0	0
1	2
2	1